

المخبر 17 | LAB 17

PINN Solution of the Swing Equation

حل معادلة التآرجح باستخدام شبكة عصبية موجّهة بالفيزياء

MATLAB Custom Training with Second-Order Automatic Differentiation

تدريب MATLAB مخصص مع اشتقاق تلقائي من الرتبة الثانية

[Dynamic Model](#)[PINN Loss](#)[Workflow](#)[MATLAB Code](#)[Validation](#)[Tasks](#)

Prepared by | إعداد | Dr.-Ing. Aouss Gabash

Laboratory Objectives

- Build a PINN that approximates rotor angle $\delta(t)$.
- Compute first- and second-order derivatives using automatic differentiation.
- Minimize swing-equation and initial-condition residuals.
- Compare the PINN with an ODE45 reference.
- Analyze loss weights, collocation points, and training stability.
- Extend the formulation to inverse parameter estimation.

أهداف المخبر

- بناء PINN لتقريب زاوية الدوار $\delta(t)$.
- حساب المشتقات الأولى والثانية تلقائيًا.
- تقليل متبقي معادلة التآرجح والشروط الابتدائية.
- المقارنة مع الحل المرجعي ODE45.
- تحليل أوزان الخسارة ونقاط الفحص واستقرار التدريب.
- توسيع النموذج إلى تقدير المعاملات.

1. Requirements

- MATLAB
- Deep Learning Toolbox

The code uses `dlnetwork` , `dlgradient` with higher derivatives, and `adamupdate` .

1. المتطلبات

يلزم MATLAB مع Deep Learning Toolbox لدعم الشبكات الديناميكية والاشتقاق التلقائي والتدريب المخصص.

2. Dynamic Model

$$M\delta''(t) + D\delta'(t) = P_m - P_{max}\sin\delta(t)$$

$$\delta(0) = \delta_0, \delta'(0) = \omega_0$$

The neural network receives time t and outputs an approximation $\tilde{\delta}(t)$.

The governing differential equation supplies supervision even when labeled angle trajectories are unavailable.

2. النموذج الديناميكي

تدخل قيمة الزمن إلى الشبكة وتنتج زاوية الدوار، ثم تُحسب مشتقات الخرج وتُستخدم داخل معادلة التآرجح.

3. Physics-Informed Loss

$$r(t) = M\delta'' + D\delta' - P_m + P_{max}\sin\delta$$

$$J_{physics} = \text{mean}[r(t)^2]$$

$$JIC = [\hat{\delta}(0) - \delta_0]^2 + [\hat{\delta}'(0) - \omega_0]^2$$

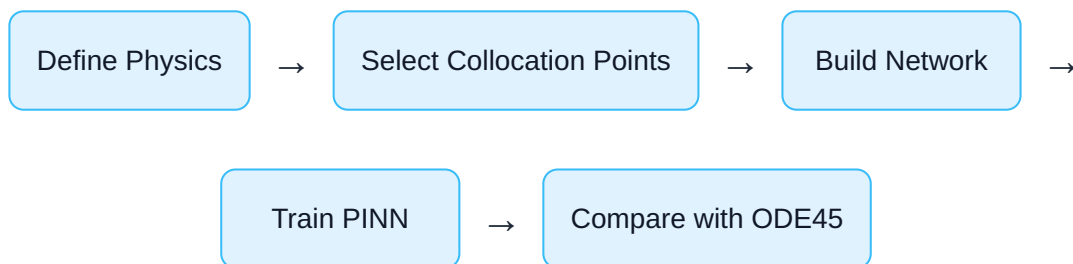
$$J = \lambda_p J_{physics} + \lambda_\delta J_{\delta 0} + \lambda_\omega J_{\omega 0}$$

Poor loss weighting can satisfy the differential equation while violating the initial conditions, or the opposite.

3. الخسارة الموجهة بالفيزياء

تجمع الخسارة بين متبقي المعادلة وخطأ زاوية البداية وخطأ السرعة الابتدائية، ويجب موازنة الأوزان بعناية.

4. Engineering Workflow



1. Define the swing equation and initial conditions.
2. Sample time-domain collocation points.
3. Construct a smooth neural network using tanh activations.
4. Compute first and second derivatives by automatic differentiation.
5. Minimize physics and initial-condition losses.
6. Validate trajectory error and physical residuals.

4. سير العمل الهندسي

نحدد المعادلة ونقاط الفحص وبنية الشبكة، ثم ندرب PINN ونقارن الحل مع ODE45 ونفحص المتبقي الفيزيائي.

5. Complete MATLAB Script

```

%% Lab 17: PINN for the Swing Equation
clear; clc; close all; rng(17);

%% 1) Physical parameters
M = 4.0; D = 1.0; Pm = 0.80; Pmax = 1.20;
delta0 = 0.30; omega0 = 0.00;
tMin = 0; tMax = 10;

%% 2) Collocation points
numPoints = 500;
tData = linspace(tMin,tMax,numPoints);
dLT = dltarray(tData,"CB");

%% 3) Neural network
layers = [
    featureInputLayer(1,Normalization="none",Name="time")
    fullyConnectedLayer(32,Name="fc1")
    tanhLayer(Name="tanh1")
    fullyConnectedLayer(32,Name="fc2")
    tanhLayer(Name="tanh2")
    fullyConnectedLayer(32,Name="fc3")
    tanhLayer(Name="tanh3")
    fullyConnectedLayer(1,Name="delta")
];
net = dlnetwork(layers);

%% 4) Training settings
numIterations = 8000;
learnRate = 1e-3;
lambdaPhysics = 1.0;
lambdaDelta0 = 50.0;
lambdaOmega0 = 50.0;
averageGrad = []; averageSqGrad = [];
lossHistory = zeros(numIterations,1);
physicsHistory = zeros(numIterations,1);
icHistory = zeros(numIterations,1);

%% 5) Custom training loop
for iteration = 1:numIterations
    [loss,gradients,lossPhysics,lossIC] = ...
        dlfeval(@modelLoss,net,dLT,M,D,Pm,Pmax, ...
            delta0,omega0,lambdaPhysics,lambdaDelta0,lambdaOmega0);

    [net,averageGrad,averageSqGrad] = adamupdate( ...
        net,gradients,averageGrad,averageSqGrad,iteration,learnRate);

```

```

        lossHistory(iteration) = double(gather(extractdata(loss)));
        physicsHistory(iteration) = double(gather(extractdata(lossPhysics)));
        icHistory(iteration) = double(gather(extractdata(lossIC)));

        if mod(iteration,500)==0
            fprintf('Iteration %5d | Total %.3e | Physics %.3e | IC %.
3e\n', ...
                iteration,lossHistory(iteration), ...
                physicsHistory(iteration),icHistory(iteration));
        end
    end

%% 6) PINN prediction
tTest = linspace(tMin,tMax,1000);
dlTTest = dlarray(tTest,"CB");
deltaPINN = extractdata(forward(net,dlTTest));

%% 7) Numerical reference
state0 = [delta0;omega0];
odefun = @(t,x) [x(2); (Pm-Pmax*sin(x(1))-D*x(2))/M];
[tODE,xODE] = ode45(odefun,[tMin tMax],state0);
deltaODE = interp1(tODE,xODE(:,1),tTest,"pchip");

%% 8) Error
MAE = mean(abs(deltaPINN-deltaODE));
RMSE = sqrt(mean((deltaPINN-deltaODE).^2));
fprintf('\nPINN MAE = %.4e rad\n',MAE);
fprintf('PINN RMSE = %.4e rad\n',RMSE);

%% 9) Plots
figure;
plot(tTest,deltaODE,"k",LineWidth=1.5); hold on;
plot(tTest,deltaPINN,"--",LineWidth=1.5); grid on;
legend("ODE45 Reference","PINN"); xlabel("Time (s)");
ylabel("Rotor Angle delta (rad)");

figure;
semilogy(lossHistory,LineWidth=1.2); hold on;
semilogy(physicsHistory,LineWidth=1.2);
semilogy(icHistory,LineWidth=1.2); grid on;
legend("Total","Physics","Initial Conditions");
xlabel("Iteration"); ylabel("Loss");

%% Local loss function

```

```

function [loss,gradients,lossPhysics,lossIC] = ...
    modelLoss(net,t,M,D,Pm,Pmax,delta0,omega0, ...
        lambdaPhysics,lambdaDelta0,lambda0mega0)

    delta = forward(net,t);
    delta_t = dlgradient(sum(delta,"all"),t, ...
        EnableHigherDerivatives=true);
    delta_tt = dlgradient(sum(delta_t,"all"),t, ...
        EnableHigherDerivatives=true);

    residual = M*delta_tt + D*delta_t - Pm + Pmax*sin(delta);
    lossPhysics = mean(residual.^2,"all");

    t0 = dlarray(0,"CB");
    deltaAt0 = forward(net,t0);
    delta0_t = dlgradient(sum(deltaAt0,"all"),t0, ...
        EnableHigherDerivatives=true);

    lossDelta0 = (deltaAt0-delta0).^2;
    loss0mega0 = (delta0_t-omega0).^2;
    lossIC = lossDelta0+loss0mega0;

    loss = lambdaPhysics*lossPhysics ...
        + lambdaDelta0*lossDelta0 ...
        + lambda0mega0*loss0mega0;
    gradients = dlgradient(loss,net.Learnables);
end

```


5. كود MATLAB الكامل

يبنى الكود شبكة تستقبل الزمن وتنتج زاوية الدوار، ثم يحسب المشتقتين ويصغر الخسارة الفيزيائية والشروط الابتدائية.

6. Why Higher Derivatives Are Enabled

The second derivative requires differentiating the first derivative. Therefore, the first call to `dlgradient` must preserve the derivative graph using `EnableHigherDerivatives=true`.

6. لماذا نفعل الاشتقاقات العليا؟

لأن المشتقة الثانية تتطلب اشتقاق المشتقة الأولى، يجب الاحتفاظ بالرسم الحسابي أثناء الاشتقاق الأول.

7. Expected Results and Validation

Loss Reduction

Total, physics, and IC losses should decrease.

Trajectory Match

The PINN should follow the ODE45 solution.

Residual Quality

The swing-equation residual should remain small over time.

Initial Conditions

Angle and speed at $t=0$ must be satisfied.

- Report MAE and RMSE.
- Plot the physical residual over the time domain.
- Compare 100, 500, and 2000 collocation points.
- Repeat training with several random seeds.
- Inspect failure near rapid transients.

Low trajectory error alone is insufficient if the differential-equation residual remains large.

7. النتائج المتوقعة والتحقق

يجب أن تنخفض الخسائر ويتبع الحل مسار ODE45، مع تحقق الشروط الابتدائية وانخفاض المتبقي الفيزيائي.

8. Student Tasks

1. Change inertia M from 4 to 2 and 8.
2. Change damping D .
3. Increase disturbance severity by changing P_m .
4. Compare 100, 500, and 2000 collocation points.
5. Test different loss weights.
6. Add 10 noisy measured angle samples.
7. Replace $\tanh$ with another activation and compare.

8. مهام الطالب

1. تغيير العطالة M .
2. تغيير التخميد D .
3. زيادة شدة الاضطراب.
4. مقارنة أعداد مختلفة من نقاط الفحص.
5. اختبار أوزان خسارة مختلفة.
6. إضافة قياسات زاوية مشوشة.
7. تغيير دالة التنشيط والمقارنة.



Research Extension

Inverse and multi-machine PINN: make M or D learnable from sparse measurements, then extend the formulation to several coupled generators and compare one shared network with one network per machine.

امتداد بحثي

اجعل العطالة أو التخميد قابلاً للتعلم من قياسات محدودة، ثم وسّع النموذج إلى عدة مولدات مترابطة.



Review Questions

1. What is the swing-equation residual?
2. Why are tanh activations useful in PINNs?
3. Why is second-order automatic differentiation required?
4. How do loss weights affect training?
5. What are collocation points?
6. Why compare against ODE45?
7. How is an inverse PINN formulated?

أسئلة المراجعة

1. ما متبقي معادلة التأرجح؟
2. لماذا تناسب tanh شبكات PINN؟
3. لماذا نحتاج اشتقاقاً تلقائياً من الرتبة الثانية؟
4. كيف تؤثر أوزان الخسارة؟
5. ما نقاط الفحص؟
6. لماذا نقارن مع ODE45؟
7. كيف تصاغ مسألة PINN عكسية؟

References | المراجع

المراجع العامة المستخدمة في هذا المقرر

- [1] A. Gabash, *Flexible Optimal Operations of Energy Supply Networks with Renewable Energy Generation and Battery Storage*. Saarbrücken, Germany: Südwestdeutscher Verlag für Hochschulschriften, 2014.
- [2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th Global ed. Harlow, U.K.: Pearson Education Limited, 2022.
- [3] A. Gabash, "Energy Market Transition and Climate Change: A Review of TSOs-DSOs C+++ Framework from 1800 to Present," *Energies*, vol. 16, no. 17, Art. no. 6139, 2023, doi: 10.3390/en16176139.
- [4] A. Gabash, *AI Applications in Electrical Power Systems*, Version 1.0, 2026. [Online]. Available: <https://aoussgabash.com>

Recommended citation:

Gabash, A. (2026). PINN Solution of the Swing Equation. *AI Applications in Electrical Power Systems*, Laboratory 17, Version 1.0. <https://aoussgabash.com>

المرجع المقترح:

أوس غباش، 2026، مخبر 17، تطبيقات الذكاء الاصطناعي في أنظمة الطاقة الكهربائية، الإصدار 1.0، <https://aoussgabash.com>

© 2026 Dr.-Ing. Aouss Gabash. Educational use with attribution to the author and official website.